

Escape Sequence Handling

Problem

Create a C program that recognizes the backslash \ as an escape character.

Creating Child Process and Executing Command

This program demonstrates:

`fork()`

`exec()`

`wait()`

```
}
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano skill6.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc skill6.c -o skill6
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./skill6
Enter text: hiiiiiiiiiiiiiiiiiiiiiii
Parsed Output: hiiiiiiiiiiiiiiiiiiiiiii
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat skill6.c
#include <stdio.h>

int main() {
    char input[200];
    int i;

    printf("Enter text: ");
    fgets(input, sizeof(input), stdin);

    printf("Parsed Output: ");

    for (i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\') {
            i++;

            if (input[i] != '\0') {
                printf("%c", input[i]);
            }
        }
        else {
            printf("%c", input[i]);
        }
    }

    return 0;
}
saikrishna__90156@DESKTOP-2JF9H5T:~$ |
```

```

}
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano skill6a.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc skill6a.c -o skill6a
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./skill6a
Parent process started.
Child process created.
Child executing: ls -l
total 332
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16176 Sep 28 07:09 Practical11
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 762 Sep 28 07:09 Practical11.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16832 Sep 29 05:01 Practical12
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 1645 Sep 29 05:01 Practical12.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16344 Sep 27 16:51 Practical5
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 1152 Sep 27 16:36 Practical5.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16392 Sep 28 05:02 Practical6
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 872 Sep 28 05:02 Practical6.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16432 Sep 28 05:05 Practical6B
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 776 Sep 28 05:04 Practical6B.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16304 Sep 28 05:15 Practical7A
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 937 Sep 28 05:15 Practical7A.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16272 Sep 28 05:16 Practical7B
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 698 Sep 28 05:16 Practical7B.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16216 Sep 28 05:32 Practical8A
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 1111 Sep 28 05:32 Practical8A.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16296 Sep 28 05:34 Practical8B
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 1131 Sep 28 05:34 Practical8B.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16264 Sep 28 05:51 Practical9A
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 866 Sep 28 05:50 Practical9A.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16264 Sep 28 05:52 Practical9B
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 784 Sep 28 05:52 Practical9B.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16128 Sep 28 05:53 Practical9C
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 885 Sep 28 05:53 Practical9C.c
prw-r--r-- 1 saikrishna__90156 saikrishna__90156 0 Sep 28 05:05 client_to_server
prw-r--r-- 1 saikrishna__90156 saikrishna__90156 0 Sep 28 05:05 server_to_client
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16136 Sep 29 05:23 skill5
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 420 Sep 29 05:22 skill5.c
-rwxr-xr-x 1 saikrishna__90156 saikrishna__90156 16136 Sep 29 05:24 skill5a

```

```
-rw-r--r-- 1 saikrishna__90156 saikrishna__90156 604 Sep 29 05:36 skill6a.c
Child process completed.
Parent process completed.
```

```
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat skill6a.c
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent process started.\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process created.\n");

        printf("Child executing: ls -l\n");

        execlp("ls", "ls", "-l", NULL);

        perror("exec failed");
        exit(1);
    }

    else {
        wait(NULL);

        printf("Child process completed.\n");
    }
}
```